

Skills de API · Tipos de autenticación · Manejo de errores · Webhooks · Google Workspace · n8n como puente

3

GUÍA 2 — MÓDULO 3

Conexión a APIs

Conectá tu agente a cualquier servicio externo — de forma real

I — Las tres formas de conectar una API en OpenClaw

OpenClaw tiene tres mecanismos distintos para conectarse con servicios externos. Cada uno sirve para una dirección del flujo diferente. Entender cuál usar antes de empezar ahorra horas de configuración incorrecta.

Mecanismo	Dirección del flujo	Para qué sirve	Cuándo usarlo
Skill de API	Agente → API externa	El agente llama a un servicio para obtener datos o ejecutar acciones.	Cuando el agente necesita consultar o actuar sobre un sistema externo: clima, CRM, base de datos, Google Calendar, etc.
Webhook	Sistema externo → Agente	Un sistema externo notifica a OpenClaw de que algo ocurrió.	Cuando querés que el agente reaccione a eventos externos: nuevo email, pago completado, alerta de servidor, commit en GitHub.
Plugin	Gateway interno	Extensión profunda del Gateway escrita en TypeScript o JavaScript.	Cuando necesitás un nuevo canal de comunicación o lógica que no puede expresarse en un skill. Requiere desarrollo.

La combinación más común en producción es skill + webhook: el webhook despierta al agente cuando llega un evento, y el skill ejecuta las acciones de respuesta sobre la API externa.

Skills de API — el agente llama al mundo

II — Construir un skill de API

Un skill de API es un archivo SKILL.md que le explica al agente cómo hablar con un servicio externo: qué URL usar, cómo autenticarse, qué parámetros enviar y cómo interpretar la respuesta. El agente lo lee cuando lo necesita y ejecuta las llamadas usando sus herramientas de shell.

Estructura base de un SKILL.md de API

```
# SKILL.md — integración con cualquier API REST
$ ---
$ nombre: nombre-del-servicio
$ descripcion: >
  Usá este skill cuando el usuario pida información o acciones
  relacionadas con [nombre del servicio]. Cubre [lista de casos].
$ ---

# [Nombre del servicio] — Referencia de API

## Autenticación
$ Tipo: Bearer Token
$ Header: Authorization: Bearer $VARIABLE_DE_ENTORNO

## URL base
$ https://api.servicio.com/v1

## Endpoints principales

### Obtener lista de recursos
$ GET /recursos
$ Header: Authorization: Bearer $TOKEN
$ Respuesta: lista de objetos con id, nombre, estado

### Crear un recurso
$ POST /recursos
$ Header: Authorization: Bearer $TOKEN
$ Header: Content-Type: application/json
$ Body: {"nombre": "valor", "tipo": "valor"}

## Manejo de errores
$ 401 — Token inválido o vencido. Notificar al usuario.
$ 429 — Límite de rate. Esperar 60 segundos y reintentar.
$ 500 — Error del servidor. Reintentar una vez. Si persiste, notificar.
```

→ Por qué el skill no guarda credenciales

El SKILL.md referencia variables de entorno (\$VARIABLE) en lugar de tokens reales. Las credenciales viven en el archivo .env del Gateway, no en el skill. Así el skill puede compartirse sin exponer información sensible, y las credenciales se cambian en un solo lugar

cuando rotan.

III — Tipos de autenticación

Las APIs usan distintos métodos de autenticación. En OpenClaw, el skill documenta el método y el agente lo ejecuta. Estos son los más frecuentes y cómo configurar cada uno.

API Key — el más simple

Un token fijo que se incluye en cada request. Sin vencimiento por defecto, pero puede ser revocado manualmente. Es el método más común en APIs de terceros.

```
# Tres formas de enviar un API key — depende de la API:

# 1. Como header (lo más común):
$ curl -H "Authorization: Bearer $MI_API_KEY" https://api.servicio.com/datos

# 2. Como header personalizado:
$ curl -H "X-API-Key: $MI_API_KEY" https://api.servicio.com/datos

# 3. Como parámetro en la URL (menos seguro, evitar si es posible):
$ curl "https://api.servicio.com/datos?api_key=$MI_API_KEY"

# En el SKILL.md documentar cuál de los tres usa el servicio
# El agente elige el formato correcto según la documentación del skill
```

OAuth — el más complejo

OAuth requiere un flujo de autorización del usuario antes de poder llamar a la API. OpenClaw maneja OAuth de forma nativa para Google, GitHub y otros servicios comunes. Para APIs personalizadas, el flujo se documenta en el skill.

```
# OAuth ya configurado en OpenClaw — Google Workspace:
$ openclaw auth google conectar

# Esto abre el navegador, el usuario autoriza,
# OpenClaw guarda el refresh token automáticamente

# Para APIs con OAuth personalizado — documentar en SKILL.md:
## Autenticación OAuth
# Paso 1 — Obtener código de autorización:
# Redirigir al usuario a: https://api.servicio.com/oauth/authorize
#   ?client_id=$CLIENT_ID
#   &redirect_uri=https://tudominio.com/callback
#   &scope=leer_datos escribir_datos

# Paso 2 — Intercambiar código por token:
$ curl -X POST https://api.servicio.com/oauth/token
```

```
-d "grant_type=authorization_code"
-d "code=CODIGO_RECIBIDO"
-d "client_id=$CLIENT_ID"
-d "client_secret=$CLIENT_SECRET"
```

```
# El access_token vence – usar el refresh_token para renovar
```

Basic Auth — legacy pero frecuente

```
# Basic Auth: usuario y contraseña codificados en Base64
# Algunas APIs antiguas o internas todavía lo usan

$ curl -u "$USUARIO:$CONTRASENA" https://api.servicio.com/datos

# O con el header explícito:
$ curl -H "Authorization: Basic $(echo -n $USUARIO:$CONTRASENA | base64)"
  https://api.servicio.com/datos
```

Tipo de auth	Complejidad	Vencimiento	Cuándo usarlo
API Key (Bearer)	Baja	No vence (se revoca)	SaaS, servicios de terceros, APIs públicas
API Key (header custom)	Baja	No vence	APIs que definen su propio nombre de header
OAuth 2.0	Alta	Access token vence (refresh automático)	Google, GitHub, servicios con datos de usuario
Basic Auth	Muy baja	No vence	APIs internas, sistemas legacy

Manejo de errores y retries — APIs que fallan

IV — Manejo de errores en el SKILL.md

Las APIs fallan. El rate limit se supera, los servidores tienen picos de carga, los tokens vencen. Si el skill no documenta cómo manejar estos casos, el agente improvisa — y suele hacerlo mal.

Códigos de error comunes y qué hacer

Código HTTP	Qué significa	Respuesta correcta del agente
400 Bad Request	El request está mal formado — parámetro faltante o formato incorrecto.	Revisar los parámetros, corregir y reintentar una vez. Si persiste, notificar al usuario con el detalle del error.
401 Unauthorized	Token inválido, vencido o ausente.	Para API Key: notificar al usuario que revise la configuración. Para OAuth: intentar refresh del token automáticamente.
403 Forbidden	El token es válido pero no tiene permisos para ese recurso.	Notificar al usuario — este error no se resuelve reintentando.
404 Not Found	El recurso no existe.	Verificar que el ID o la URL sean correctos. Notificar al usuario si el recurso realmente no existe.
429 Too Many Requests	Se superó el límite de requests por tiempo.	Esperar el tiempo indicado en el header Retry-After (o 60 segundos si no está) y reintentar.
500 / 503	Error del servidor externo.	Reintentar una vez después de 30 segundos. Si persiste, notificar al usuario y dejar de reintentar.

Política de retries en el SKILL.md

```
# En la sección de manejo de errores del SKILL.md:

## Política de reintentos
$ - Errores 4xx (excepto 429): NO reintentar. Son errores del request, no del servidor.
$ - Error 429: Esperar el tiempo del header Retry-After. Si no está presente, esperar 60s.
  Reintentar máximo 2 veces. Si persiste, notificar al usuario.
$ - Errores 5xx: Esperar 30 segundos. Reintentar máximo 1 vez.
  Si persiste, notificar al usuario con el código de error.
$ - Timeout (sin respuesta en 30s): Reintentar una vez. Si vuelve a fallar, notificar.
```

```
## Qué incluir en la notificación al usuario
$ - Qué acción se estaba intentando
$ - El código de error recibido
$ - Si ya se reintentó y cuántas veces
$ - Qué puede hacer el usuario para resolverlo
```

⚠ El agente no debe reintentar indefinidamente

Sin una política de reintentos documentada en el skill, el agente puede entrar en un loop costoso. Definir siempre un máximo de reintentos y una condición de salida clara. Un agente que reintentó 10 veces por un error 403 está gastando tokens sin sentido y nunca va a resolver el problema.

Webhooks — el mundo externo llama al agente

V — Configurar webhooks en OpenClaw

Un webhook es un endpoint HTTP que OpenClaw expone para recibir notificaciones de sistemas externos. Cuando GitHub hace un push, cuando Stripe procesa un pago, cuando un servidor genera una alerta — esos sistemas pueden llamar al endpoint de OpenClaw y despertar al agente.

Activar el endpoint de webhooks

```
# En openclaw.json — sección hooks:
"hooks": {
  "habilitado": true,
  "token": "tu-token-secreto-largo-y-aleatorio",
  "ruta": "/hooks",
  "agentesPermitidos": ["agente-principal", "agente-equipo"]
}

# El endpoint queda disponible en:
# http://tu-vps:18789/hooks

# IMPORTANTE: proteger con nginx + HTTPS antes de exponerlo
# Nunca exponer el puerto del Gateway directamente al internet
```

Dos modos de webhook

Modo	Endpoint	Para qué sirve	Ejemplo
Wake (despertar)	/hooks/wake	Notifica al agente de que ocurrió algo. El agente ejecuta su próximo heartbeat o una tarea específica.	Alerta de servidor caído — despierta al agente para que verifique el estado
Agent run (ejecutar)	/hooks/agent	Envía un mensaje completo al agente y puede entregar la respuesta de vuelta a un canal.	Nuevo ticket de soporte — el agente clasifica, resume y responde

Llamada de prueba al endpoint

```
# Probar el modo wake:
$ curl -X POST http://tu-vps:18789/hooks/wake
```

```
-H "Authorization: Bearer tu-token-secreto"
-H "Content-Type: application/json"
-d '{"texto": "Nuevo evento recibido", "modo": "ahora"}'

# Probar el modo agent run:
$ curl -X POST http://tu-vps:18789/hooks/agente
-H "x-openclaw-token: tu-token-secreto"
-H "Content-Type: application/json"
-d '{"mensaje": "Resumí la bandeja de entrada", "agenteId": "agente-principal",
"modoDespertar": "ahora"}'
```

⚠ Los webhooks reintentan — el agente debe ser idempotente

Todos los sistemas de webhooks bien diseñados reintentan cuando no reciben confirmación (HTTP 200). Eso significa que tu agente puede recibir el mismo evento dos o tres veces. Si la acción es 'crear una tarea', crearla dos veces es un problema.

La solución es usar una clave de sesión fija por evento (sessionKey) en la configuración del webhook. OpenClaw no procesa un segundo mensaje con la misma sessionKey si ya tiene esa sesión activa. Así los reintentos son descartados automáticamente.

Google Workspace — el caso más común

VI — Integración con Google Workspace

Google Calendar, Gmail y Google Drive son las integraciones más usadas en la comunidad de OpenClaw. OpenClaw las soporta de forma nativa con skills oficiales y un flujo de autenticación OAuth integrado.

Autenticación inicial

```
# Conectar Google una sola vez:
$ openclaw auth google conectar

# Abre el navegador → autorizar con tu cuenta Google
# OpenClaw guarda el refresh token automáticamente
# El access token se renueva solo cuando vence

# Verificar que la conexión está activa:
$ openclaw auth google estado
```

Skills disponibles

Skill	Qué puede hacer el agente
google-calendar	Crear, leer, modificar y eliminar eventos. Buscar disponibilidad. Crear recordatorios. Invitar participantes.
gmail	Leer, buscar y redactar emails. Marcar como leído. Mover a carpetas. Crear borradores. Responder hilos.
google-drive	Leer y buscar archivos. Crear documentos y hojas de cálculo. Compartir con permisos específicos. Descargar contenido.

Integración de Gmail via webhook — el flujo completo

El caso de uso más documentado: Gmail notifica a OpenClaw cuando llega un email nuevo, el agente lo procesa y toma acción. Esto requiere configurar Gmail Pub/Sub.

```
# Paso 1 — Configurar Gmail para publicar en Pub/Sub:
$ openclaw webhooks gmail configurar

# Esto configura Gmail para que notifique al endpoint de OpenClaw
# cada vez que llega un email nuevo a la bandeja configurada
```

```
# Paso 2 — El webhook recibe la notificación:
# Gmail → POST https://tu-vps/hooks/wake
# OpenClaw despierta al agente

# Paso 3 — El agente procesa con el skill de Gmail:
# Lee el email nuevo
# Clasifica, resume o responde según AGENTS.md

# Ejemplo de regla en AGENTS.md:
## Procesamiento de emails
# Cuando se recibe un email nuevo:
# - Si es de un cliente conocido: prioridad alta, notificar en WhatsApp
# - Si es newsletter: archivar sin notificar
# - Si contiene "urgente": responder con acuse de recibo y alertar
```

VII — n8n como puente entre APIs y OpenClaw

n8n es una herramienta de automatización que se complementa muy bien con OpenClaw. La división de roles es clara: n8n maneja las integraciones técnicas, transformaciones de datos y credenciales de APIs complejas. OpenClaw recibe el resultado y aplica razonamiento.

n8n hace bien	OpenClaw hace bien
Guardar y gestionar credenciales de múltiples APIs	Razonar sobre datos y decidir qué hacer
Transformar datos entre formatos distintos	Redactar respuestas y resúmenes en lenguaje natural
Manejar flujos con muchos pasos secuenciales	Clasificar, priorizar y tomar decisiones complejas
Reintentar y manejar errores de API sin gastar tokens	Generar contenido, responder usuarios, ejecutar acciones finales

Patrón A — n8n despierta a OpenClaw

n8n recibe el evento externo (nuevo lead en el CRM, pago en Stripe, alerta de monitoreo), hace el procesamiento técnico y luego llama al webhook de OpenClaw con un mensaje ya preparado.

```
# n8n hace el trabajo técnico y llama a OpenClaw:
# Nodo HTTP Request en n8n:
$ URL: https://tu-vps/hooks/agente
$ Método: POST
$ Headers:
  x-openclaw-token: tu-token-secreto
  Content-Type: application/json
$ Body:
{
  "mensaje": "Nuevo cliente registrado: {{$json.nombre}}, plan {{$json.plan}}. Enviar email de bienvenida personalizado.",
  "agenteId": "agente-principal",
  "modoDespertar": "ahora",
  "entregar": true,
  "canal": "ultimo"
}
```

Patrón B — OpenClaw llama a n8n

El agente usa un skill para hacer una llamada POST al webhook de n8n cuando necesita ejecutar un flujo complejo que involucra múltiples APIs o transformaciones que no vale la pena delegar al modelo.

```
# En el SKILL.md del agente – skill para llamar a n8n:
$ ---
$ nombre: disparar-flujo-n8n
$ descripcion: Usá este skill cuando necesites ejecutar un flujo de automatización
  que involucre múltiples sistemas. Disponible: sincronizar-crm, generar-reporte,
  procesar-pagos.
$ ---

## Cómo disparar un flujo
$ POST https://tu-n8n.dominio.com/webhook/NOMBRE-DEL-FLUJO
$ Header: Authorization: Bearer $N8N_WEBHOOK_TOKEN
$ Body: {"datos": "a procesar", "origen": "openclaw"}

## Flujos disponibles
$ - sincronizar-crm: actualiza contactos en HubSpot desde datos de conversación
$ - generar-reporte: produce Excel con datos del mes y lo sube a Drive
$ - procesar-pagos: verifica estado de cobros en Stripe y genera resumen
```

Casos reales — integraciones documentadas en producción

VIII — Casos reales de integración

Caso 1 — Agente que monitorea un servidor

```
# Flujo completo:
# 1. Sistema de monitoreo detecta error 500 en el servidor
# 2. Llama al webhook de OpenClaw con los datos del error
# 3. OpenClaw despierta al agente
# 4. El agente usa su skill de shell para revisar los logs
# 5. Clasifica la gravedad del error
# 6. Si es crítico: notifica por WhatsApp inmediatamente
# 7. Si es leve: registra en MEMORY.md y monitorea

# Webhook de entrada:
$ curl -X POST https://tu-vps/hooks/agente
-H "x-openclaw-token: SECRET"
-d '{"mensaje": "Error 500 en /api/pagos. Logs: [...]", "agenteId": "agente-devops"}'
```

Caso 2 — Agente que gestiona leads del CRM

```
# Flujo:
# 1. Nuevo lead llega al CRM (HubSpot, Salesforce, etc.)
# 2. CRM llama al webhook de OpenClaw
# 3. El agente lee los datos del lead
# 4. Clasifica: frío / tibio / caliente según criterios en AGENTS.md
# 5. Si es caliente: notifica al usuario por WhatsApp con resumen
# 6. Usa skill de Gmail para enviar email de bienvenida personalizado
# 7. Registra en MEMORY.md para seguimiento

# Regla en AGENTS.md:
## Clasificación de leads
# Lead caliente: empresa > 50 empleados Y presupuesto > $5000
# Lead tibio: empresa 10-50 empleados O presupuesto $1000-5000
# Lead frío: resto
```

Caso 3 — Agente que publica contenido

```
# Flujo de publicación automatizada:
# 1. Heartbeat diario a las 8AM
# 2. Agente lee noticias via skill de RSS
# 3. Selecciona las 3 más relevantes para la audiencia (definida en SOUL.md)
```

```
# 4. Redacta un resumen en el tono del canal
# 5. Usa skill de API de red social para programar la publicación
# 6. Registra en MEMORY.md qué publicó para no repetir

# Skills necesarios:
# - rss-reader: leer feeds de noticias
# - late-api o similar: publicar en redes sociales
# - google-calendar: verificar si hay restricciones del calendario de publicaciones
```

→ **Siguiente módulo: Prompt Engineering y Diseño de Conversación**

Instrucciones que el modelo realmente sigue, flujos conversacionales complejos y cómo testear prompts en producción.

Módulo 3 · KeroClow · www.openagent.lat